

An Introduction to Programming through C++

Abhiram G. Ranade

Ch. 16: Arrays and Recursion

Arrays and Recursion

- Recursion is very useful for designing algorithms for data stored in arrays
- Topics
 - Binary Search
 - Merge Sort

Searching an array

Input:

- `A`: int array of length `n`,
- `x` (called “key”) : int

Output:

- `true` if `x` is present in `A`,
- `false` otherwise.

Natural algorithm:

Scan through the array and return true if found.

Time consuming:

- Entire array scanned if the element is not present,
- Half array scanned on the average if it is present.

```
bool search(int *A, int x,
            int n){
    for(int i=0; i<n; i++){
        if(A[i] == x)
            return true;
    }
    return false;
}
// “Linear search”
```

Searching a sorted array

- sorted array: (non decreasing order)

$$A[0] \leq A[1] \leq \dots \leq A[n-1]$$

- sorted array: (non increasing order)

$$A[0] \geq A[1] \geq \dots \geq A[n-1]$$

- How do we search in a sorted array?
 - Does the sortedness help in searching?

Searching for x in a non decreasing sorted array $A[0..n-1]$

Key idea: First compare x with the “middle” element $A[n/2]$ of the array.

Suppose $x < A[n/2]$:

- x is also smaller than $A[n/2 .. n-1]$, because of sorting
- x if present will be present only in $A[0 .. n/2 - 1]$.
- So in the rest of the algorithm we will only search first half of A .

Suppose $x \geq A[n/2]$:

- x if present will be present in $A[n/2 .. n-1]$
- Note: x may be present in first half too,
- In the rest of the algorithm we will only search second half.

How to search the “halves”?

- Recurse

In one comparison we have eliminated half the array!

Plan

We will write a function `Bsearch`

- It will search a region of an array instead of the entire array.

Region: specified using 2 numbers:

- Starting index S , length L of region

When $L == 1$, we are searching a length 1 array.

- So check if that element, $A[S] == x$.

Otherwise, compare x to the “middle” element of $A[S .. S+L-1]$

- Middle element: $A[S + L/2]$

Algorithm is called “Binary search”

- Because size of the region to be searched gets roughly halved.

The code

```
bool Bsearch(int A[], int S, int L, int x)
{ // Search x in A[S..S+L-1]
    if(L == 1) return A[S] == x;
    int H = L/2;
    if(x < A[S+H])
        return Bsearch(A, S,    H,    x); // region A[S..S+H-1]
    else
        return Bsearch(A, S+H, L-H, x); // region A[S+H.. S+L-1]
}

int main(){
    int A[8 ]={-1, 2, 2, 4, 10, 12, 30, 30};
    cout << Bsearch(A,0,8,11) << endl; // searches for 11.
}
```

Let us run some basic checks..

- Is there a base case? Is the correct answer being returned for the base case?
 - Base case: $L = 1$
- Are recursive calls valid, i.e. stay within the region?
- Are the recursive calls solving “smaller” problems?
 - Problem size = L , Call 1 size = H , Call 2 size = $L-H$
- If the answer to recursive calls is correct, then will it produce the correct answer for the top level call?

What we discussed

- The search problem: decide if a certain key is present in an array.
- If array is not sorted, we do Linear Search.
- If array is sorted, we do Binary search.
- Next: Sample execution of binary search, analysis



Execution

```
bool Bsearch(int A[], int S, int L,
int x){
    if(L == 1) return A[S] == x;
    int H = L/2;
    if(x < A[S+H])
        return Bsearch(A, S, H, x);
    else
        return Bsearch(A, S+H, L-H, x);
}
```

```
int main(){
    int A[8]={-1,2,2,4,10,12,30,30};
    cout<< Bsearch(A,0,8,11) << endl;
} // searches for 11.
```

Bsearch(A, 0, 8, 11)

- S=0, L=8, x=11
- H=4
- $x < A[S+H]$?
- $11 < A[4]=10$ is false.
- Bsearch(A, S+H, L-H, x) called

Bsearch(A, 4, 4, 11)

- S=4, L=4, x=11
- H=2
- $x < A[S+H]$?
- $11 < A[6]=30$ is true
- Bsearch(A, S, H, x) called

Bsearch(A, 4, 2, 11)

Execution continued

```
bool Bsearch(int A[], int S, int L,
int x){
    if(L == 1) return A[S] == x;
    int H = L/2;
    if(x < A[S+H])
        return Bsearch(A, S, H, x);
    else
        return Bsearch(A, S+H, L-H, x);
}
```

```
int main(){
    int A[8]={-1,2,2,4,10,12,30,30};
    cout<< Bsearch(A,0,8,11) << endl;
} // searches for 11.
```

Bsearch(A, 4, 2, 11)

- S=4, L=2, x=11
- H=1
- $x < A[S+H]$?
- $11 < A[5]=12$ is true.
- Bsearch(A, S, H, x) called.

Bsearch(A, 4, 1, 11)

- S=4, L=1, x=11
- L==1 is true
- Return $11 == A[5]$.
- So returns false.
- Returned all the way back to main.

Demo

- Bsearch.cpp

Remarks

Even professional programmers make mistakes when writing binary search.

- Should condition be $x \leq A[H]$ or $x < A[H]$?
- Need to ensure correctness even if length L is odd.
- Subranges to be searched have to be specified carefully
- Very important to write down precisely what the function does: “searches $A[S..S+L-1]$ ” – be careful about -1 etc.

Is it worth going to the trouble?

- Is binary search better than linear search, i.e. going through the elements from left to right?
- Consider an array of size 1000.
- In second call, we search an array of size 500.
- In third call, we search an array of 250.
- ...
- In tenth call, we search an array of size 1. So return.
- Time taken: 10 calls worth, v.s. 1000 comparisons.
- Time is proportional to $\log N$, vs. N .

What we discussed

- Binary search is a very fast way to search for a key in a sorted array.
- You may observe some similarity with bisection method.
- If you are likely to search an array frequently,
 - First sort it, then search.
 - The time to sort the array will be compensated by the time saved in subsequent searches.
- How do you sort an array in the first place? **Next.**



Sorting

- Sorting:

Input: array containing numbers

Goal: rearrange numbers in non-decreasing order

- Selection Sort (Chapter 14)

- Total time: “About n^2 ”.

- Next: Merge sort.

- Total time: “About $n \log n$ ”

Mergesort idea

To sort a long sequence:

- Break up the sequence into two small sequences.
- Sort each small sequence. (Recurse!)
- Somehow “merge” the sorted sequences into a single long sequence.

Hope: “merging” sorted sequences is easier than sorting the large sequence.

- Our hope is correct, as we will see soon!

Example

- Suppose we want to sort the sequence
 - 50, 29, 87, 23, 25, 7, 64
- Break it into two sequences.
 - 50, 29, 87, 23 and 25, 7, 64.
- Sort both
 - We get 23, 29, 50, 87 and 7, 25, 64.
- Merge
 - Goal is to get 7, 23, 25, 29, 50, 64, 87.

Merge sort

```
void mergesort(int S[], int n){           // Sorts sequence S of length n.
    if(n==1) return;

    int U[n/2], V[n-n/2];                // local arrays
    for(int i=0; i<n/2; i++)
        U[i]=S[i];                       // copy of first half
    for(int i=0; i<n-n/2; i++)
        V[i]=S[i+n/2];                   // copy of second half

    mergesort(U,n/2);
    mergesort(V,n-n/2);

    merge(U, n/2, V, n-n/2, S);          // merge into original array S.
}
```

Exercise

The merge function will be discussed next. But assume it is available. Show how mergesort will sort the sequence 50, 29, 87, 23, 25, 7, 64

- Draw the recursion tree for mergesort.
- Show the sequence that is the argument to each call and the sequence returned by each call.

What we discussed

- Outline of mergesort
 - Split given sequence into 2 parts
 - Sort the 2 parts recursively
 - Merge the 2 sorted sequences
- Code for mergesort function

Next: the merge function



Merging example

U: 23, 29, 50, 87.

V: 7, 25, 64.

S:

- The smallest overall must move into S.
- Smallest overall = smaller of smallest in U and smallest in V.
- So after movement we get:

U: 23, 29, 50, 87.

V: -, 25, 64.

S: 7.

What do we do next?

U: 23, 29, 50, 87.

V: -, 25, 64.

S: 7.

- Now we need to move the second smallest into S.
- Second smallest:
 - smallest in U,V after smallest has moved out.
 - smaller of what is at the “front” of U, V.
- So we get:

U: -, 29, 50, 87.

V: -, 25, 64.

S: 7, 23.

General strategy

- While both U, V contain a number:
 - Move smallest from those at the front of U,V to the end of S.
- If only U contains numbers: move all to end of S.
- If only V contains numbers: move all to end of S.

Elements leave an array from the “front” and join another array at the “back”

- Similar to taxi dispatch

Details

- uf : index denoting which element of U is currently at the front.
 - $U[0..uf-1]$ have moved out.
- vf : similarly for V .
- sb : (“back of s ”) index denoting where next element should move into S next
 - $S[0..sb-1]$ contain elements that have moved in earlier.
- Lengths of U, V : L_u, L_v
- Length of S : guaranteed to be $L_u + L_v$, so need not define it separately.

Merging two sequences

```
void merge(int U[], int Lu, int V[], int Lv, int S[]){
    int uf=0, vf=0;
    for(int sb=0; sb < Lu+Lv; sb++){
        if(uf<Lu && vf<Lv){                // both U,V are non empty
            if(U[uf] < V[vf])
                { S[sb] = U[uf]; uf++;}
            else
                { S[sb] = V[vf]; vf++;}
        }
        else if(uf < Lu)                    // only U is non empty
            {S[sb] = U[uf]; uf++;}
        else                                // only V is non empty
            {S[sb] = V[vf]; vf++;}
    }
}
```

Time Analysis: merging

Time required to merge two sequences of length L_u , L_v :

- Loop runs for $L_u + L_v$ iterations.
- In each iteration a fixed number of operations are performed.
- So time is proportional to $L_u + L_v$.
- Time proportional to n , if $n = L_u + L_v$.

What we discussed

- The function to merge sorted sequences
- The time required to merge two sequences of total length n is proportional to n .

Next:

- Putting together everything: analysis of entire mergesort algorithm and a demo
- Summing up for entire lecture



Time analysis: mergesort

```
void mergesort(int S[], int n){
    if(n==1) return;
    int U[n/2], V[n-n/2];
    for(int i=0; i<n/2; i++)
        U[i]=S[i];
    for(int i=0; i<n-n/2; i++)
        V[i]=S[i+n/2];
    mergesort(U,n/2);
    mergesort(V,n-n/2);
    merge(U, n/2, V, n-n/2, S, n);
}
```

- T_i = maximum time required for mergesort to sort any sequence of length i .

$T_1 \leq c$, where c is some constant.

$$T_n \leq dn + 2T_{n/2} + en$$

- create U, V from S + sort halves + merge

$$T_n \leq fn + 2T_{n/2} \text{ for } f=d+e$$

- Inequality applies to $T_{n/2}$ also

$$T_{n/2} \leq fn/2 + 2T_{n/4}$$

$$T_n \leq fn + 2(fn/2 + 2T_{n/4}) = 2fn + 4T_{n/4}$$

- Continuing we get

$$T_n \leq kfn + 2^k T_{n/2^k}$$

- If $n=2^k$ or $k = \log_2 n$:

$$T_n \leq fn \log n + nT_1 = fn \log_2 n + nc$$

- Thus $T_n \leq gn \log_2 n$ for some constant g .

Same applies even if n is not a power of 2...

Demo

- mergesort.cpp

Concluding remarks

- Binary search is much faster than linear search.
- Mergesort is much faster than selection sort.
- In both algorithms, we divide original problem into smaller problems: “divide-and-conquer”
- Recursion is very useful with arrays.

